

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING



ET3291E - Design project I

Building a Snake Game with Java Swing

Student: Bùi Thanh Thảo

Student ID: 20233876

Instructor: PhD. Vương Hoàng Nam

Hanoi, 06/2026.

Table of Contents

LIST OF FIGURES	4
INTRODUCTION	5
CHAPTER 1. PROJECT OVERVIEW	6
1.1 Problem Statement	6
1.2 Objectives and Scope	6
1.3 Technology Overview	7
CHAPTER 2. THEORETICAL BACKGROUND	8
2.1 Java Programming Language	8
2.2 Object-Oriented Programming (OOP)	8
a) Encapsulation	8
b) Inheritance	8
c) Interface and Polymorphism	9
d) Single Responsibility Principle	9
2.3 Java Swing Library	9
CHAPTER 3. REQUIREMENTS ANALYSIS	10
3.1 Problem Description	10
3.2 Functional Requirements	10
3.3 Non-Functional Requirements	11
CHAPTER 4. SYSTEM DESIGN	13
4.1 Overall Architecture	13
4.2 OOP Design	14
4.2.1 Encapsulation in ScoreManager	14
4.2.2 Interface Callback Pattern	15
4.2.3 CardLayout Navigation Pattern	15

4.3 System Activity Flow.....	15
CHAPTER 5. IMPLEMENTATION	16
5.1 Development Environment	16
5.2 GameMode Enum	16
5.3 ScoreManager Class.....	16
5.4 StartPanel Class	17
5.5 GameFrame Class	18
5.6 GamePanel Class	19
5.6.1 Snake Data Structure.....	19
5.6.2 Game Loop	19
5.6.3 Collision Detection by Mode.....	20
5.6.4 Random Wall Generation (WALL mode).....	21
CHAPTER 6. TESTING AND RESULTS	23
6.1 Functional Testing	23
6.2 Result Evaluation.....	24
Strengths.....	24
Limitations.....	25
CHAPTER 7. CONCLUSION	26
7.1 Summary	26
7.2 Limitations.....	26
7.3 Future Development.....	27
REFERENCES	28

LIST OF FIGURES

Figure 4.1	System architecture of the Snake Game	13
Figure 5.1	Start Screen interface showing three game modes	17
Figure 5.2	Game board in CLASSIC mode during gameplay	19
Figure 5.3	WALL mode with randomly generated obstacles	20
Figure 5.4	Game Over screen displaying NEW RECORD badge	21
Figure 5.5	PAUSED overlay screen	21

INTRODUCTION

In the process of studying programming, building real applications plays an important role in reinforcing theoretical knowledge and developing problem-solving skills. Simple games provide an ideal environment for applying learned concepts in practice, especially in the areas of object-oriented programming and graphical user interface design.

The Snake game is one of the most well-known classic games, with simple rules that require quick reflexes and strategic thinking. Its use of array-based data structures, game loops, keyboard event handling, collision detection, and graphical rendering makes it a suitable topic to implement using Java and the Swing GUI library.

This project, "Building a Snake Game with Java Swing," aims to create a fully functional game application with a clean user interface and stable behavior. Beyond the core mechanics, the project extends to include three distinct game modes, a Start screen, pause functionality, dynamic speed scaling, and persistent high score storage.

Through this project, the author aims to strengthen skills in object-oriented design, Swing-based GUI development, and software architecture, while producing a complete and presentable deliverable for academic evaluation.

CHAPTER 1. PROJECT OVERVIEW

1.1 Problem Statement

Object-oriented programming (OOP) is the foundation of most modern software systems. However, mastering OOP through theory alone is often insufficient for students to deeply understand concepts such as encapsulation, inheritance, polymorphism, and system design. Building a complete application from scratch gives students a direct experience of the full software development cycle: analysis, design, implementation, and testing.

The Snake game was selected as the project topic because of its universal familiarity, straightforward rules, and sufficient technical complexity to demonstrate core programming concepts including data structures, game loops, event handling, collision detection, state management, and graphical rendering.

1.2 Objectives and Scope

The objectives of this project are to build a Snake game application running on the Java Desktop platform with the following requirements:

- Build a Snake game with a graphical user interface (GUI) using Java Swing.
- Allow the player to control the snake using arrow keys or WASD.
- Generate food randomly; increase snake length and score when food is eaten.
- Detect collisions with walls and the snake's own body.
- Display score and a Game Over screen with restart functionality.
- Extend with three game modes, pause support, speed scaling, and persistent high score storage.

The scope of implementation includes:

- A Snake game running on Windows with JDK 11 or later.
- Interface built exclusively with Java Swing; no external graphics libraries.
- Three game modes: CLASSIC (border kills), ENDLESS (wrap-around), and WALL (random obstacles).

- High score stored per game mode in a local file named `highscore.txt`.

1.3 Technology Overview

- Java SE (JDK 25): Primary programming language with full OOP support.
- Java Swing (`javax.swing`): Java's built-in GUI library for building desktop interfaces.
- `java.awt`: 2D graphics support (`Graphics`, `Color`, `Font`, `FontMetrics`).
- `javax.swing.Timer`: Thread-safe timer for driving the game loop at fixed intervals.
- `java.io` (`BufferedReader`, `BufferedWriter`): File I/O for reading and writing `highscore.txt`.
- `java.util.Random`: Generates random positions for food and wall segments.
- `java.util.ArrayList`: Stores wall tile coordinates in WALL mode.

CHAPTER 2. THEORETICAL BACKGROUND

2.1 Java Programming Language

Java is an object-oriented programming language developed by Sun Microsystems in 1995 and currently maintained by Oracle. Java achieves platform independence through the Java Virtual Machine (JVM): source code is compiled into bytecode (.class files), which the JVM then executes on any supported operating system.

Key Java features applied in this project include:

- Strong typing: Explicit type declarations reduce runtime errors.
- Garbage collection: Automatic memory management eliminates manual deallocation.
- Exception handling: try-catch blocks in ScoreManager handle file I/O errors gracefully.
- Enum types: The GameMode enum enforces type safety for the three game modes.
- Switch expressions: Used in Java 14+ for concise mode-based branching.

2.2 Object-Oriented Programming (OOP)

Object-oriented programming organizes code around objects rather than functions and logic alone. The four core principles of OOP are applied throughout the project as follows.

a) Encapsulation

Each class hides its internal data and exposes only a well-defined public interface. For example, ScoreManager fully encapsulates file I/O logic. GamePanel calls only getHighScore(mode) and saveIfBetter(mode, score) without knowing how data is stored on disk. Similarly, GamePanel encapsulates all game state (snake coordinates, food position, walls, flags) and exposes no direct field access to outside classes.

b) Inheritance

GamePanel extends JPanel to gain the ability to override paintComponent() for custom rendering. GameFrame extends JFrame to become the main application window. StartPanel also extends JPanel for the Start screen UI. This demonstrates how Java inheritance enables code reuse while allowing specialized behavior.

c) Interface and Polymorphism

GamePanel implements ActionListener so it can receive events from javax.swing.Timer directly. The inner interface GameStartListener, defined in StartPanel, declares a single method onGameStart(GameMode). GameFrame implements this interface, allowing StartPanel to notify the frame when the player clicks Play without creating a direct dependency between the two classes. This is a classic example of the callback pattern and loose coupling in OOP.

d) Single Responsibility Principle

Each class is responsible for exactly one concern: GamePanel handles game logic and rendering; ScoreManager handles persistent storage; StartPanel handles the mode selection UI; GameFrame handles screen navigation. This separation makes the codebase easier to read, test, and extend.

2.3 Java Swing Library

Swing is Java's built-in GUI toolkit, part of the Java Foundation Classes (JFC). It provides a rich set of components and allows full graphical customization by overriding paintComponent(Graphics g).

Swing components used in this project:

- JFrame: Top-level application window (GameFrame).
- JPanel: Drawable container with custom painting (GamePanel, StartPanel).
- JButton: Clickable buttons in the Start screen.
- JLabel: Text labels for mode descriptions and hints.
- GridBagLayout: Flexible layout manager used in StartPanel for centered vertical alignment.

- CardLayout: Container layout in GameFrame that switches between StartPanel and GamePanel.
- javax.swing.Timer: Fires ActionEvents on the EDT at fixed intervals, serving as the game loop.

A critical note: Swing is single-threaded. All UI operations must run on the Event Dispatch Thread (EDT). Using javax.swing.Timer instead of a raw Thread ensures that actionPerformed() is always called on the EDT, making it safe to update the game state and repaint the panel without synchronization overhead.

CHAPTER 3. REQUIREMENTS ANALYSIS

3.1 Problem Description

The Snake game is played on a 600 x 600 pixel board divided into a 24 x 24 grid of 25 x 25 pixel tiles. The player controls a snake that moves continuously in one direction. The snake grows longer each time it eats a food item (a red dot), and the score increases by 10 points per food eaten. The game ends when the snake collides with a wall (in CLASSIC mode) or with its own body.

This project extends the basic Snake game with three distinct game modes that alter collision behavior at the board boundary, a dynamic speed system that increases difficulty as the score rises, and a persistent high score system that records the best result per mode across sessions.

3.2 Functional Requirements

The functional requirements of the system are summarized in the table below.

No.	Feature	Description
1	Start Screen	Displays game title, three mode buttons, Play button, and control hints

No.	Feature	Description
2	Mode Selection	Three modes: CLASSIC (border kills), ENDLESS (wrap-around), WALL (obstacles)
3	Snake Movement	Controlled via arrow keys or WASD; 180-degree reversal is blocked
4	Game Loop	Timer fires every 150 ms; each tick moves the snake and redraws the board
5	Food Generation	Random tile not overlapping the snake body or wall segments
6	Eating Food	Snake grows by one tile, score increases by 10, speed adjusts
7	Collision Detection	Wall collision (CLASSIC), self-collision (all modes), random wall (WALL)
8	Score Display	Shows current Score, Best Score, and Level during gameplay
9	Speed Scaling	Delay decreases by 10 ms every 50 points; minimum delay is 40 ms
10	Pause	P or ESC toggles pause; semi-transparent overlay keeps board visible
11	Game Over Screen	Shows score, record, level reached, and restart instructions
12	High Score Storage	ScoreManager writes highscore.txt per mode; NEW RECORD badge on beat
13	Restart	R key resets all state for a new game without reopening the window
14	Return to Menu	M key stops the timer and returns to StartPanel at any time

3.3 Non-Functional Requirements

- Performance: Game loop runs stably at 150 ms per tick with no visible lag or stuttering.
- Usability: The interface is self-explanatory; no manual is needed to understand the controls.
- Extensibility: Clear class separation allows new features to be added without modifying existing logic.
- Reliability: ScoreManager handles file errors gracefully; no crash occurs on corrupt or missing files.
- Compatibility: Runs on Windows with JDK 11 or later using only the Java Standard Library.

CHAPTER 4. SYSTEM DESIGN

4.1 Overall Architecture

The system is designed using an object-oriented approach and divided into six classes, each with a clearly defined responsibility.

Class	Responsibility	Extends / Implements
Main	Entry point; starts the application on the EDT	N/A
GameFrame	Main window; manages screen navigation	JFrame, GameStartListener
GameMode	Defines the three game modes as constants	Enum
StartPanel	Start screen UI; mode selection and Play button	JPanel
ScoreManager	Reads and writes high scores to highscore.txt	N/A
GamePanel	Game logic, rendering, input handling, and state	JPanel, ActionListener

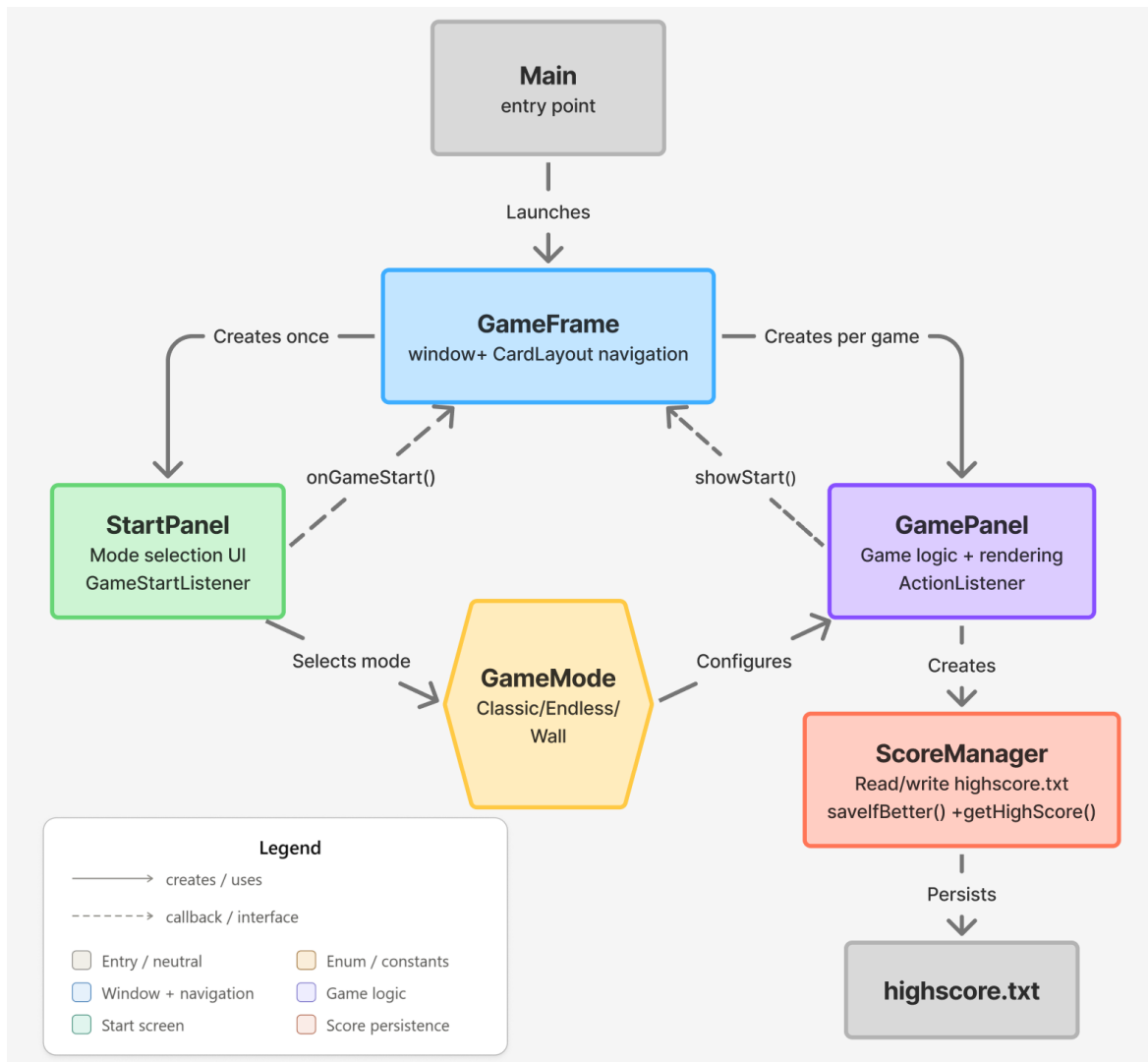


Figure 4.1 System architecture of the Snake Game

The dependency flow between classes is as follows: Main creates GameFrame. GameFrame holds StartPanel and GamePanel through a CardLayout container. StartPanel notifies GameFrame via the GameStartListener interface. GamePanel uses ScoreManager to persist scores and holds a reference to GameFrame for screen navigation.

4.2 OOP Design

4.2.1 Encapsulation in ScoreManager

ScoreManager stores high scores in a HashMap with GameMode as the key and Integer as the value. It exposes three public methods: getHighScore(GameMode), saveIfBetter(GameMode, int), and the constructor which loads the file. GamePanel

interacts only through these methods and is completely unaware of the file path, format, or parsing logic.

4.2.2 Interface Callback Pattern

StartPanel declares a nested interface GameStartListener with one method: `onGameStart(GameMode mode)`. GameFrame implements this interface. When the player clicks Play, StartPanel calls `listener.onGameStart(selectedMode)`. StartPanel does not hold any reference to GameFrame directly; it only knows the listener interface. This achieves loose coupling between the two screens.

4.2.3 CardLayout Navigation Pattern

GameFrame wraps both screens inside a single JPanel using CardLayout. Each time the player starts a new game, a fresh GamePanel instance is created with the selected mode and added to the container. This guarantees complete state reset and avoids bugs from leftover state carried between game sessions.

4.3 System Activity Flow

The complete activity flow of the application proceeds as follows:

- `Main.main()` calls `SwingUtilities.invokeLater()` to create GameFrame on the EDT.
- GameFrame displays StartPanel (card "start") by default.
- The player selects a mode and clicks PLAY; StartPanel fires `listener.onGameStart(mode)`.
- GameFrame creates a new GamePanel with the selected mode and switches to card "game".
- `GamePanel.startGame()` calls `initSnake()`, then `generateWalls()` if mode is WALL, then `spawnFood()`, then `Timer.start()`.
- Every 150 ms, the Timer fires `actionPerformed()`. If `isRunning` and not `isPaused`, `move()` is called followed by `repaint()`.
- `move()` shifts body segments, updates the head, calls `checkFood()`, then calls `checkCollision()`.

- On game over, `triggerGameOver()` calls `ScoreManager.saveIfBetter()` and repaints the Game Over overlay.
- Pressing R calls `startGame()` for a new round. Pressing M stops the timer and calls `GameFrame.showStart()`.

CHAPTER 5. IMPLEMENTATION

5.1 Development Environment

- Language: Java SE (JDK 25.0.2)
- IDE: VS Code with Extension Pack for Java / IntelliJ IDEA Community Edition
- Operating System: Windows 10 / Windows 11
- Libraries: Java Standard Library only (no external dependencies)
- Build: Direct `javac` compilation from the command line; no build tool required

Compile and run commands:

```
javac Main.java GameFrame.java GameMode.java StartPanel.java
ScoreManager.java GamePanel.java
java Main
```

5.2 GameMode Enum

`GameMode` is an enum that defines the three game modes as named constants. Using an enum instead of integer or string constants provides compiler-checked exhaustiveness in switch expressions: if a new mode is added but a switch block does not handle it, the compiler raises an error. This prevents silent bugs from missing cases.

```
public enum GameMode {
    CLASSIC,    // Border is active; hitting the wall ends the game
    ENDLESS,    // No border; the snake wraps to the opposite side
    WALL        // No border; random wall segments kill on contact
}
```

5.3 ScoreManager Class

ScoreManager manages high scores keyed by GameMode and persists them in a plain-text file named `highscore.txt` located in the working directory. The file uses a simple key-value format:

```
CLASSIC=120  
ENDLESS=200  
WALL=80
```

The constructor calls `load()` immediately to populate the internal `HashMap`. `load()` opens the file with `BufferedReader`, splits each line on the `"="` character, converts the mode string using `GameMode.valueOf()`, and parses the score with `Integer.parseInt()`. If the file does not exist (first run) or any line is malformed, the exception is caught silently and the default value of 0 is kept. This ensures the application never crashes due to a missing or corrupt file.

`saveIfBetter(GameMode, int)` compares the new score against the stored value before writing. If the new score is higher, the map is updated and `saveToFile()` overwrites the entire file using `BufferedWriter`. The method returns a boolean indicating whether a new record was set, which `GamePanel` uses to display the **NEW RECORD** badge.

5.4 StartPanel Class

`StartPanel` extends `JPanel` and uses `GridBagLayout` to vertically center all UI components. The layout contains: a large green "SNAKE" title label, a subtitle, three mode selection buttons, a dynamic description label, a Play button, and a control hint at the bottom.

A key technical detail is the prevention of button size changes when the border color is swapped on selection. The solution requires calling `setPreferredSize()`, `setMinimumSize()`, and `setMaximumSize()` with the same `Dimension` object (440 x 54 pixels) on every button, and setting `GridBagConstraints.fill` to `NONE` so `GridBagLayout` does not stretch the component. Additionally, `RoundedBorder` overrides `getBorderInsets()` to always return a fixed `Insets` value regardless of the border color, preventing any layout recalculation when selection state changes.

RoundedBorder is an inner class that extends AbstractBorder. Its paintBorder() method draws a rounded rectangle using Graphics2D.drawRoundRect() with antialiasing enabled for smooth edges. When a mode is selected, the button's background changes to dark green and the border color changes to bright green; all other buttons revert to the default gray style.

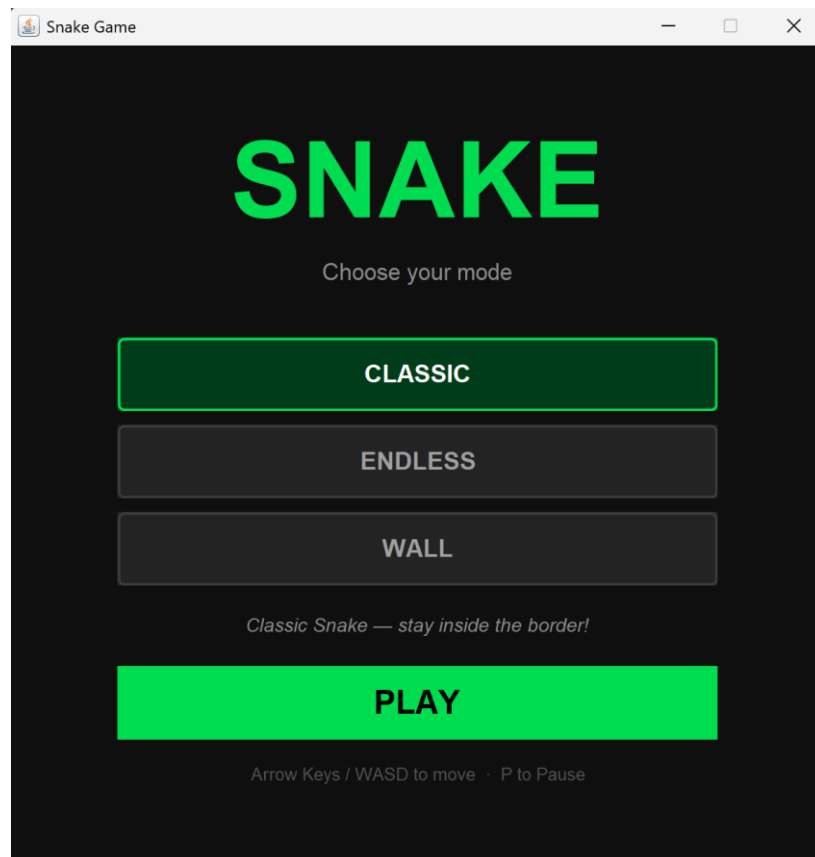


Figure 5.1 Start Screen interface showing three game modes

5.5 GameFrame Class

GameFrame extends JFrame and implements GameStartListener. The constructor creates a CardLayout container, adds StartPanel as card "start", and calls showStart() to display the Start screen. GamePanel is not created in the constructor; it is instantiated only when the player clicks Play, ensuring the mode is known before construction.

showGame(GameMode) removes any existing GamePanel from the container (to free resources and avoid duplicate Timer instances), creates a new GamePanel with the chosen mode, adds it as card "game", and switches the CardLayout. showStart()

reverses this by switching back to card "start" and returning focus to StartPanel for keyboard input.

5.6 GamePanel Class

GamePanel is the central class of the application. It extends JPanel, implements ActionListener, and contains all game state, rendering logic, and input handling. The key constants that define the game board are:

```
TILE_SIZE = 25 px      GRID_WIDTH = 24      GRID_HEIGHT = 24
INITIAL_DELAY = 150 ms  MIN_DELAY = 40 ms    SPEED_STEP = 10 ms
SCORE_PER_LEVEL = 50
WALL_COUNT = 8          WALL_MIN_LEN = 3      WALL_MAX_LEN = 5    SAFE_ZONE:
tiles (8,8) to (16,16)
```

5.6.1 Snake Data Structure

The snake is represented by two parallel integer arrays, snakeX[] and snakeY[], each of size MAX_TILES = 576 (24 x 24). Index 0 is always the head. An integer snakeLength tracks how many tiles are currently alive. This approach avoids object creation per frame (unlike ArrayList<Point>) and enables O(1) access to any tile by index.

The movement algorithm per tick proceeds in two steps: first, shift all body segments backward from the tail toward index 1 (for i = snakeLength - 1 down to 1: pos[i] = pos[i-1]); second, update pos[0] according to the current direction. The backward iteration is essential to avoid overwriting a position before it has been copied to the next index. When food is eaten, snakeLength is simply incremented: the tile previously at index snakeLength becomes active without any new allocation.

5.6.2 Game Loop

The game loop is implemented with javax.swing.Timer. The constructor creates a Timer with INITIAL_DELAY as the interval and "this" (GamePanel) as the ActionListener. Every tick, actionPerformed() checks isRunning && !isPaused before calling move() and repaint(). This means the Timer continues to run even during pause

or Game Over, but has no effect until the flags are cleared. This avoids the complexity of stopping and restarting the Timer for every state transition.

Dynamic speed is computed as: $\text{newDelay} = \max(\text{INITIAL_DELAY} - (\text{score} / \text{SCORE_PER_LEVEL}) * \text{SPEED_STEP}, \text{MIN_DELAY})$. `adjustSpeed()` calls `gameTimer.getDelay()` and only applies `setDelay()` and `restart()` when the value actually changes, avoiding unnecessary Timer resets on every food pick-up.

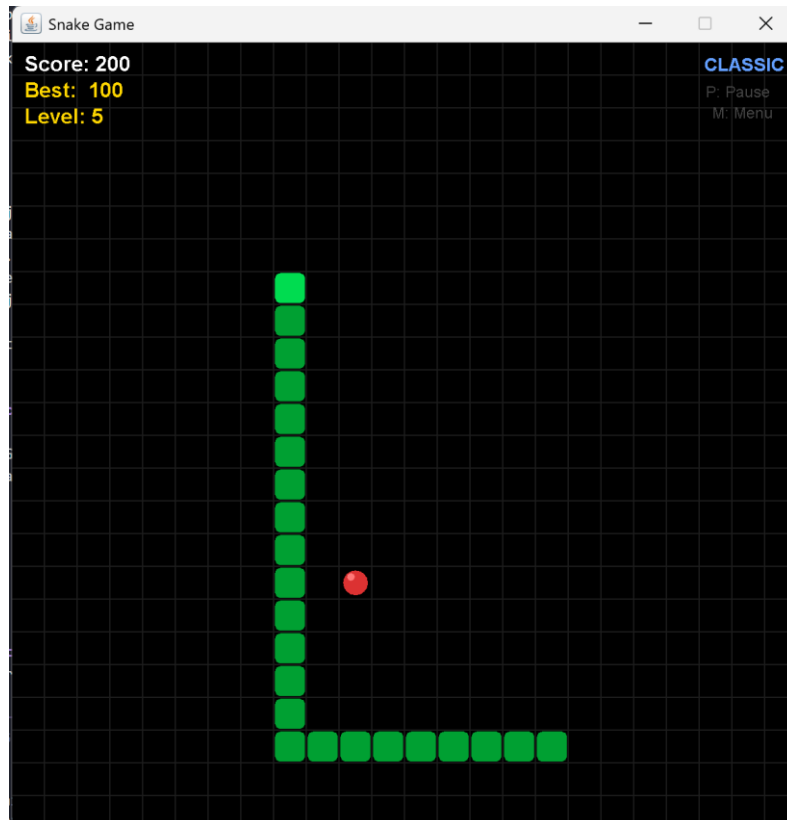


Figure 5.2 Game board in CLASSIC mode during gameplay

5.6.3 Collision Detection by Mode

`checkCollision()` branches on the current GameMode after each move:

- CLASSIC: `hitBorder()` checks whether `snakeX[0]` or `snakeY[0]` is out of the `[0, GRID_WIDTH)` or `[0, GRID_HEIGHT)` range. If true, `triggerGameOver()` is called.
- ENDLESS: `wrapAround()` maps out-of-bound coordinates back to the opposite side: `x = -1` becomes `x = GRID_WIDTH - 1`; `x = GRID_WIDTH` becomes `x = 0`; and similarly for Y. The snake never dies from the border.

- WALL: wrapAround() handles the border first, then hitWall() iterates the walls ArrayList to check whether snakeX[0] and snakeY[0] match any wall tile. If true, triggerGameOver() is called.

Self-collision is checked last and applies to all three modes: a loop from $i = 1$ to $\text{snakeLength} - 1$ compares each body tile to the head position. Starting at index 1 is critical; checking index 0 would always match the head against itself.

5.6.4 Random Wall Generation (WALL mode)

generateWalls() creates $\text{WALL_COUNT} = 8$ wall segments. For each segment, a random direction (horizontal or vertical), a random length between WALL_MIN_LEN and WALL_MAX_LEN , and a random origin point are chosen. The origin is clamped so the entire segment stays within the board. Each tile of the proposed segment is then checked against two conditions: it must not fall inside the safe zone (tiles 8 to 16 in both X and Y) and it must not overlap any already-placed wall tile. If a conflict is found, the entire attempt is discarded and a new one is tried. A maximum of 200 total attempts prevents an infinite loop if the board becomes too crowded.

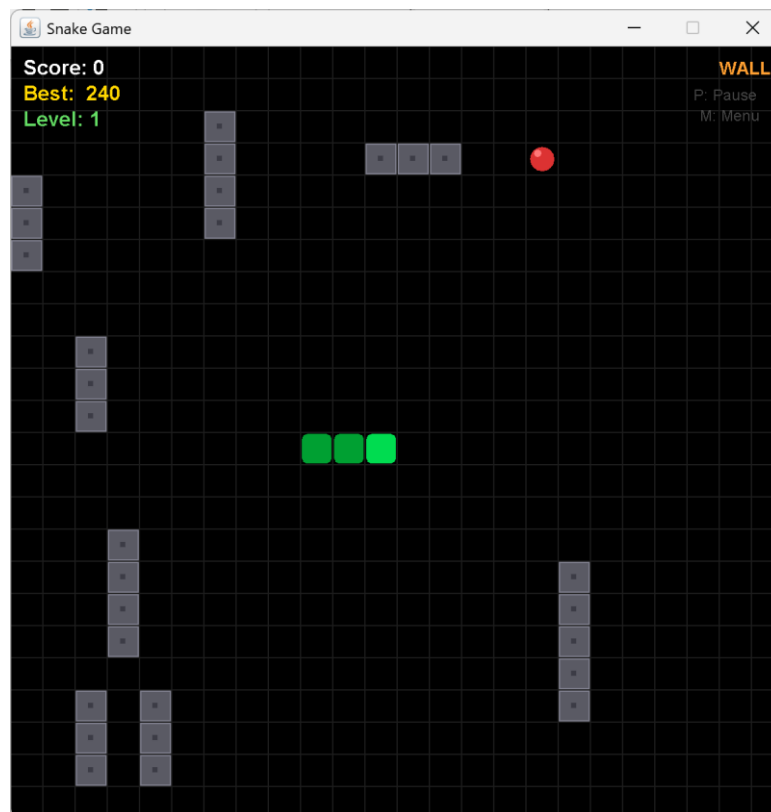


Figure 5.3 WALL mode with randomly generated obstacles

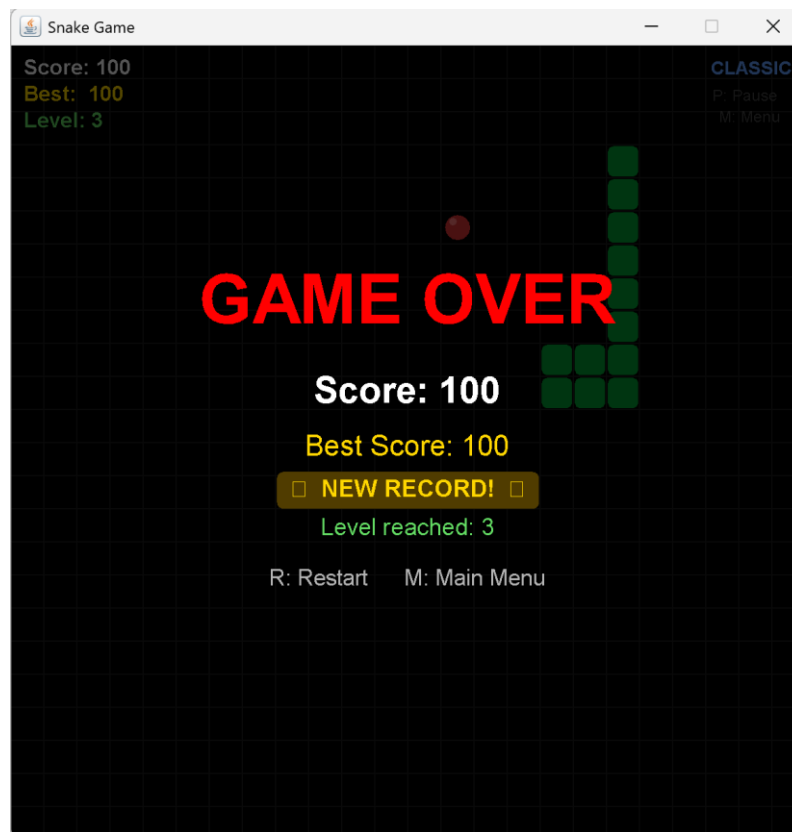


Figure 5.4 Game Over screen displaying NEW RECORD badge

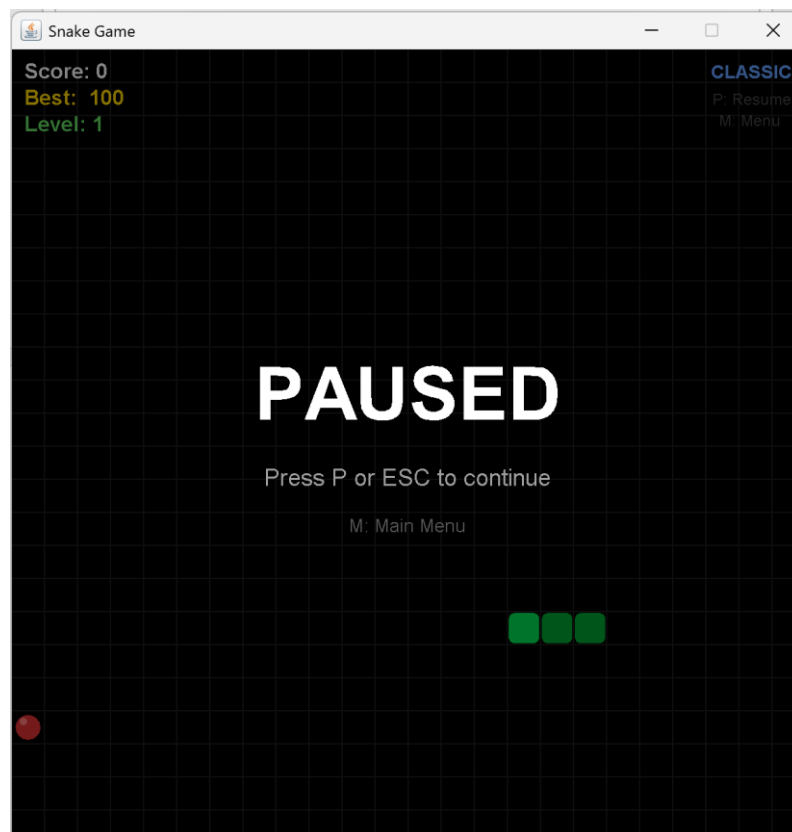


Figure 5.5 PAUSED overlay screen

CHAPTER 6. TESTING AND RESULTS

6.1 Functional Testing

All functional requirements were tested manually against defined test cases. The purpose of testing was to confirm that each feature behaves as specified, identify and resolve defects during development, and validate the stability of the game under continuous play.

No.	Feature Under Test	Test Scenario	Expected Result	Outcome
1	Start Screen	Launch application	Three mode buttons visible; CLASSIC highlighted by default	PASS
2	Mode selection	Click each button	Selected button highlights; description label updates	PASS
3	Game loop start	Click PLAY	Snake moves right automatically every 150 ms	PASS
4	Direction control	Press arrow keys and WASD	Snake changes direction; 180-degree reversal blocked	PASS
5	Food consumption	Steer snake onto food tile	Snake grows by one, score increases by 10, new food appears	PASS
6	Speed scaling	Reach 50, 100, 150 points	Delay decreases 10 ms per 50 pts; Level indicator updates	PASS
7	Wall collision CLASSIC	Steer snake into border	Game Over triggered immediately	PASS
8	Wrap-around ENDLESS	Steer snake past any edge	Snake reappears on the opposite side	PASS

No.	Feature Under Test	Test Scenario	Expected Result	Outcome
9	WALL obstacle hit	Steer snake into a gray wall tile	Game Over triggered	PASS
10	Self-collision	Steer snake into its own body	Game Over triggered	PASS
11	Pause	Press P or ESC during play	Snake stops; PAUSED overlay appears; board still visible	PASS
12	Resume from pause	Press P or ESC while paused	Game resumes at the same speed and position	PASS
13	High score persistence	Beat record, close, reopen	highscore.txt updated; record visible on next session	PASS
14	NEW RECORD badge	Achieve a new best score	Golden badge appears on Game Over screen	PASS
15	Restart	Press R after Game Over	All state resets; WALL mode regenerates new wall layout	PASS
16	Return to menu	Press M at any time	Timer stops; Start Screen displayed with mode selection intact	PASS

6.2 Result Evaluation

Strengths

- All 16 test cases pass with consistent behavior across multiple play sessions.
- The three game modes are clearly differentiated in both mechanics and visual style.

- Dynamic speed scaling creates a natural difficulty curve that increases player engagement.
- ScoreManager persists records reliably; no data loss occurs on application close or file corruption.
- The codebase follows a clear separation of concerns: six classes with distinct, independent roles.
- The application is stable: no crashes observed under any tested input sequence.

Limitations

- The best score (bestScore) shown during gameplay resets when the player returns to the menu, because GamePanel is recreated on each new game. The persistent record in highscore.txt is correctly maintained, but the in-session best is lost.
- There are no sound effects: no audio feedback on eating food, Game Over, or record-breaking.
- Snake movement is tile-by-tick with no interpolation, which appears as discrete jumps rather than smooth sliding at high speeds.
- WALL mode does not increase obstacle count with level; difficulty comes only from speed, not new walls.

CHAPTER 7. CONCLUSION

7.1 Summary

After the full design and implementation cycle, the Snake Game project has been completed and meets all stated objectives. The application delivers a fully functional desktop game with three distinct modes, a polished Start screen, dynamic speed progression, pause support, a detailed Game Over screen, and file-based high score persistence.

From a technical perspective, the project successfully demonstrates the following programming concepts: object-oriented design with six well-separated classes, inheritance (JPanel, JFrame), the interface callback pattern (GameStartListener), event-driven programming (KeyAdapter, ActionListener), array-based data structures for the snake, collision detection algorithms, file I/O, and GUI construction with Java Swing including custom painting, CardLayout navigation, and custom border rendering.

Through this project, the author has strengthened practical skills in:

- Object-oriented design: defining class responsibilities, interfaces, and enums.
- Java Swing GUI development: custom painting, layout managers, and component styling.
- System thinking: decomposing a problem into independent modules and designing their interactions.
- Debugging and testing: identifying and resolving layout bugs, timer issues, and collision edge cases.

7.2 Limitations

- The in-session best score is lost when the player returns to the menu, because GamePanel is newly instantiated each game. The file-persisted record is correct, but the displayed best during gameplay does not carry over.

- No sound effects are present. Java's `javax.sound.sampled` API introduces platform-specific compatibility concerns on Windows, so audio was excluded from scope.
- The board size and window dimensions are fixed at 600 x 600 pixels. The UI does not adapt to different screen resolutions.
- WALL mode obstacle density is static. At very high levels (delay below 60 ms), certain random wall configurations may create unavoidable dead ends.

7.3 Future Development

Based on the identified limitations and potential improvements, the following directions are proposed for future development:

- Pass `bestScore` from the previous `GamePanel` to the new instance via `GameFrame` so the in-session best is preserved across restarts.
- Add short audio clips (food pickup, Game Over, new record) using `javax.sound.sampled.Clip` loaded from bundled `.wav` resources.
- Implement a top-5 leaderboard per mode in `highscore.txt`, replacing the single-record system.
- Scale WALL mode obstacle count with level: add one extra wall segment every 100 points to increase spatial difficulty independently of speed.
- Add a two-player mode on the same keyboard: one player uses WASD, the other uses arrow keys.
- Introduce special food items that appear for a limited time and award bonus points when eaten.

REFERENCES

- [1] K. Sierra and B. Bates, Head First Java, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.
- [2] H. Schildt, Java: The Complete Reference, 12th ed. New York, NY, USA: McGraw-Hill Education, 2021.
- [3] Oracle Corporation, "Java Platform, Standard Edition Documentation," 2025. [Online]. Available: <https://docs.oracle.com/javase/>. [Accessed: Jul. 2026].
- [4] Oracle Corporation, "The Swing Tutorial," 2025. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/>. [Accessed: Jul. 2026].
- [5] GeeksforGeeks, "Java Swing Tutorial," 2025. [Online]. Available: <https://www.geeksforgeeks.org/java-swing/>. [Accessed: Jul. 2026].
- [6] Stack Overflow, "Java Swing and Event Handling Discussions," 2025. [Online]. Available: <https://stackoverflow.com/>. [Accessed: Jul. 2026].
- [7] Hanoi University of Science and Technology, "Lecture Notes: Object-Oriented Programming," Hanoi, Vietnam.